

```
=====
## What is Exception?
=====
```

=> An un-expected and un-wanted situation which distrubs normal flow of our program execution is called as Exception.

=> When exception occurs, normal program execution will stop (Abnormal termination)

=> As a programmer we are responsible to handle exceptions occuring in the programs.

=> Exception handling is used to handle runtime errors without stopping the program suddenly.

```
=====
## Simple Meaning
=====
```

Exception means runtime error.

Example :

```
a = 10
b = 0
```

```
result = a/b
```

```
print("Result : ", result)
```

Output :

ZeroDivisionError: division by zero

```
=====
## Why Exception Handling?
=====
```

Exception handling is used for:

- 1) Handling Runtime Errors
- 2) Avoid program crash
- 3) Display user-friendly error messages
- 4) Graceful Termination (Continue execution till the end)

```
=====
## Types of Errors
=====
```

- 1) Syntax Error
- 2) Runtime Error / Exception

```
=====
## Syntax Error
=====
```

** Syntax error occurs when we write wrong Python syntax.

Example:

```
print("Hello
```

Output :

```
SyntaxError: '(' was never closed
```

Import : Syntax errors can't be handled using Exception handling.

Note: We must fix syntax errors manually to start program execution.

```
=====
## Runtime Error / Exception
=====
```

```
** Runtime error occurs while program is running.
```

```
** These errors can be handled using exception handling.
```

Example :

```
a = 10
b = 0
```

```
print(a / b)
```

Output :

```
ZeroDivisionError: division by zero
```

```
=====
## Common Exceptions in Python
=====
```

- 1) ZeroDivisionError
- 2) ValueError
- 3) TypeError
- 4) IndexError
- 5) KeyError
- 6) FileNotFoundError
- 7) NameError
- 8) AttributeError
- 9) ImportError
- 10) IndentationError

```
=====
## ZeroDivisionError
=====
```

This exception occurs when we divide a number by zero.

Example :

```
a = 10
b = 0
```

```
print(a / b)
```

Output :

```
ZeroDivisionError: division by zero
```

```
=====
## ValueError
=====
```

This exception occurs when wrong value is given.

Example :

```
num = int(input("Enter value : "))
```

```
print(num)
```

```
=====
## TypeError
=====
```

This exception occurs when operation is performed on wrong data type.

Example :

```
result = "10" + 20
```

```
print(result)
```

Output :

```
TypeError: can only concatenate str not int to str
```

```
=====
## IndexError
=====
```

This exception occurs when we access invalid index from list, tuple or string.

Example :

```
numbers = [10, 20, 30]
```

```
print(numbers[5])
```

Output :

```
IndexError: list index out of range
```

```
=====
## KeyError
=====
```

This exception occurs when we access invalid key from dictionary.

Example :

```
student = {  
    "name" : "Ashok",  
    "course" : "GEN AI"  
}
```

```
print(student["age"])
```

Output :

```
KeyError: 'age'
```

```
=====
```

This exception occurs when file is not available.

Example :

```
file = open("students.txt", "r")
```

```
print(file.read())
```

Output :

```
FileNotFoundError
```

```
=====
```

This exception occurs when we use a variable that is not defined.

Example :

```
print(course)
```

Output :

```
NameError: name 'course' is not defined
```

```
=====
```

This exception occurs when we call unavailable method or attribute.

Example :

```
numbers = [10, 20, 30]
```

```
numbers.upper()
```

Output :

```
AttributeError: 'list' object has no attribute 'upper'
```

```
=====
## ImportError
=====
```

This exception occurs when import fails.

Example :

```
from math import addition
```

Output :

```
ImportError
```

```
=====
## Exception Handling Keywords
=====
```

- 1) try
- 2) except
- 3) else
- 4) finally
- 5) raise

```
=====
try and except
=====
```

**** try block is used to keep risky code. Risky code means code that may generate exception.**

**** except block is used to handle exception. If exception occurs in try block then only except block will execute. If no exception in try block then except block will not execute.**

Ex-1:

```
a = int(input("Enter First Number : "))
b = int(input("Enter Second Number : "))
```

```
try:
    result = a / b
    print("Result:", result)
```

```
except:
    print("Invalid Number Entered")
```

Ex-2:

```
try:
    a = int(input("Enter first Number:"))
    b = int(input("Enter second Number:"))

    result = a / b
```

```
    print("Result : ", result)

except ValueError as ve:
    print("Please enter only numbers : ", ve)

except ZeroDivisionError as ze:
    print("Don't enter zeros : ", ze)

except Exception as e:
    print("Some Problem Occured : ", e)

print("Program Execution Completed")
```

Note: Always handle specific exceptions first. Generic Exception should be written at the end.

```
=====
else block
=====
```

** else block executes only when no exception occurs in try block.

Syntax :

```
try:
    risky code
except:
    error handling code
else:
    code to execute when no exception occurs
```

Example :

```
try:
    num1 = int(input("Enter first Number:"))
    num2 = int(input("Enter first Number:"))
    result = num1 / num2
except ZeroDivisionError as ze:
    print("Cannot divide by zero : ", ze)
else:
    print("Result : ", result)
```

```
=====
finally block
=====
```

** finally block always executes.

** It executes whether exception occurs or not.

** finally block is mainly used for cleanup activities.

Example :

```
try:
    print(10/2)
except ZeroDivisionError as ze:
    print("Cannot divide by zero : ", ze)
finally:
    print("Program Execution Completed")
```

```
=====
raise keyword
=====
```

**** raise keyword is used to generate exception manually.**

Ex :

```
age = int(input("Enter your age : "))

if age >= 18:
    print("You are old enough to vote")
else:
    raise Exception("Age must be 18 or above to vote")
```

```
=====
## Custom Exception
=====
```

**** We can create our own exception class.**

**** Custom exception class should inherit from Exception class.**

Ex:

```
class InvalidAgeError(Exception):
    pass

try:
    age = int(input("Enter your age : "))
    if age < 18:
        raise InvalidAgeError("You are not old enough to vote")
    print("You are old enough to vote")
except InvalidAgeError as e:
    print("Invalid Age Error : ", e)
except ValueError as ve:
    print("Invalid Input: ", ve)
finally:
    print("Program Execution Completed")
```

```
=====
## Difference Between Error and Exception
=====
```

Error:

Error is a problem in the program.
Some errors are due to wrong syntax.
Syntax errors must be fixed manually.

Exception:

Exception is a runtime error.
Exception occurs during program execution.
Exceptions can be handled using try except.

```
=====
## Difference Between except and finally
=====
```

except:

except block executes only when exception occurs.
It is used to handle exception.

finally:

finally block always executes.
It is used for cleanup activities.

```
=====
## Difference Between raise and except
=====
```

raise: raise is used to create exception manually.

except: except is used to handle exception.

```
=====
## Important Points
=====
```

- ** Exception means runtime error.
- ** When Exception occurs program will terminate abnormally.
- ** Exception handling prevents sudden program crash.
- ** try block contains risky code.
- ** except block handles exception.
- ** else block executes when no exception occurs.
- ** finally block always executes.
- ** raise keyword is used to create exception manually.
- ** Exception is the parent class for most exceptions.
- ** Handle specific exceptions first.
- ** Generic Exception should be written at the end.
- ** Syntax errors cannot be handled using try except.
- ** Runtime errors can be handled using try except.
- ** finally is mainly used for cleanup activities.
- ** Custom exceptions can be created by inheriting Exception class.
- ** Do not use empty except block in real-time projects.
- ** Always show meaningful error messages.